

Lab sheet-6

Ashish singh Rawat

Cu24250221

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
# Set visualization theme
```

```
sns.set_theme(style="whitegrid")
```

```
plt.rcParams.update({'font.sans-serif': 'DejaVu Sans', 'font.size': 10})
```

```
# Step 1 & 2: Load the Dataset and Import Libraries
```

```
# Exact filename: California Housing Dataset.csv
```

```
filename = "California Housing Dataset.csv"
```

```
df = pd.read_csv(filename)
```

```
print(f"Dataset successfully loaded. Shape: {df.shape}\n")
```

```
print(df.info())
```

```
# Standardize column headers (strips spaces and converts to lowercase)
```

```
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')
```

```
# Step 3: Data Preprocessing & Feature Selection
```

```
# 1. Identify target column across naming conventions
```

```
target_candidates = ['median_house_value', 'medhouseval', 'price', 'house_value']
```

```
target_col = next((col for col in target_candidates if col in df.columns), df.columns[-1])
```

```
print(f"\nTarget Variable identified: '{target_col}'")
```

```
# 2. Impute missing values (e.g., total_bedrooms) with median
```

```
for col in df.select_dtypes(include=[np.number]).columns:
```

```
    if df[col].isnull().sum() > 0:
```

```
        median_val = df[col].median()
```

```

df[col] = df[col].fillna(median_val)

print(f"Imputed missing values in '{col}' with median: {median_val:.2f}")

# 3. Handle categorical columns (e.g., ocean_proximity) via One-Hot Encoding
categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()

if categorical_cols:
    df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
    print(f"Categorical features encoded: {categorical_cols}")

# 4. Separate Independent Features (X) and Dependent Target (y)
X = df.drop(columns=[target_col])
y = df[target_col]

# Step 4: Split into Training and Testing Sets (80:20 Ratio)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print(f"\nTraining set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples\n")

# Step 5: Train the Linear Regression Model
model = LinearRegression()
model.fit(X_train, y_train)

# Step 6: Model Predictions on Test Data
y_pred = model.predict(X_test)

# Step 7: Visualization and Automatic Plot Saving
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# Subplot 1: Actual vs. Predicted Values
axs[0].scatter(y_test, y_pred, alpha=0.25, color='#1f77b4', edgecolors='none', s=20)
axs[0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r', linewidth=2)
axs[0].set_title("Actual vs. Predicted House Value", fontsize=12, weight='bold')
axs[0].set_xlabel("Actual Values", fontsize=11)

```

```

axs[0].set_ylabel("Predicted Values", fontsize=11)

# Subplot 2: Residual Error Distribution
residuals = y_test - y_pred

sns.histplot(residuals, kde=True, bins=40, color='#2ca02c', ax=axs[1])

axs[1].axvline(0, color='red', linestyle='--', linewidth=1.5)

axs[1].set_title("Residual Error Distribution", fontsize=12, weight='bold')

axs[1].set_xlabel("Residuals (Actual - Predicted)", fontsize=11)

axs[1].set_ylabel("Frequency", fontsize=11)

plt.tight_layout()

plt.savefig("california_housing_regression_plots.png", dpi=300, bbox_inches='tight')

plt.close()

print("[SAVED] Plot exported as 'california_housing_regression_plots.png'")

# Step 8: Performance Metrics Evaluation

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("=" * 60)

print("MODEL PERFORMANCE METRICS")

print("=" * 60)

print(f"Mean Absolute Error (MAE):    {mae:,.4f}")

print(f"Mean Squared Error (MSE):    {mse:,.4f}")

print(f"Root Mean Squared Error (RMSE): {rmse:,.4f}")

print(f"R-squared Score (R²):         {r2:,.4f}")

# Step 9 & 10: Regression Coefficients & Impact Analysis

coefficients = pd.DataFrame({

    'Feature': X.columns,

    'Coefficient': model.coef_

```

```

}).sort_values(by='Coefficient', key=abs, ascending=False)

print("\n" + "=" * 60)

print("FEATURE COEFFICIENTS (IMPACT ANALYSIS)")

print("=" * 60)

print(f"Model Intercept: {model.intercept_:.4f}\n")

print(coefficients.to_string(index=False))

```



```
Dataset successfully loaded. Shape: (20640, 10)
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 20640 entries, 0 to 20639
```

```
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	longitude	20640 non-null	float64
1	latitude	20640 non-null	float64
2	housing_median_age	20640 non-null	float64
3	total_rooms	20640 non-null	float64
4	total_bedrooms	20433 non-null	float64
5	population	20640 non-null	float64
6	households	20640 non-null	float64
7	median_income	20640 non-null	float64
8	median_house_value	20640 non-null	float64
9	ocean_proximity	20640 non-null	str

```
dtypes: float64(9), str(1)
```

```
memory usage: 1.7 MB
```

```
None
```

Training set size: 16512 samples
Testing set size: 4128 samples

[SAVED] Plot exported as 'california_housing_regression_plots.png'

=====

MODEL PERFORMANCE METRICS

=====

Mean Absolute Error (MAE): 50,670.7382
Mean Squared Error (MSE): 4,908,476,721.1566
Root Mean Squared Error (RMSE): 70,060.5218
R-squared Score (R²): 0.6254

=====

FEATURE COEFFICIENTS (IMPACT ANALYSIS)

=====

Model Intercept: -2,275,547.3817

	Feature	Coefficient
	ocean_proximity_ISLAND	136125.072615
	ocean_proximity_INLAND	-39786.656161
	median_income	39473.975175
	longitude	-26838.273372
	latitude	-25468.352050
	ocean_proximity_NEAR BAY	-5136.642217
	ocean_proximity_NEAR OCEAN	3431.140073
	housing_median_age	1102.185084
	total_bedrooms	102.789395
	households	48.252753
	population	-38.172906
	total_rooms	-6.021506